HDOC Day-54

Aim

To write a C program to find the **maximum element in every contiguous subarray (sliding window) of size `k`** using pointers and a separate function `window_maximum()` with a **nested-loop approach**.

Algorithm

1. Start the program.
2. Read the size of the array.
3. Declare a variable-length integer array `arr[size]`.
4. Read the elements of the array.
5. Read the value of `k`.
6. Pass the array to `window_maximum()` using its address.
7. Check whether `k` is valid.
    - If `k <= 0` or `k > size`, display `-1`.
8. Using the outer loop, consider every contiguous window of size `k`.
9. For each window, assume its first element is the maximum.
10. Using the inner loop, compare all the remaining elements of that window.
11. Update the maximum whenever a greater element is found.
12. Display the maximum element of each window separated by spaces.
13. Stop.

C Program

```
#include <stdio.h>

void window_maximum(int *arr, int size, int k)
{
    int i, j, max;

    if (k <= 0 || k > size)
    {
        printf("-1\n");
        return;
    }

    // Consider each window of size k
    for (i = 0; i <= size - k; i++)
    {
        max = *(arr + i);
```

```
        // Find maximum element in current window
        for (j = 1; j < k; j++)
        {
            if (*(arr + i + j) > max)
            {
                max = *(arr + i + j);
            }
        }

        printf("%d", max);

        if (i < size - k)
        {
            printf(" ");
        }
    }

    printf("\n");
}

int main()
{
    int size, i, k;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    int arr[size];

    printf("Enter the array elements: ");
    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: ");
    scanf("%d", &k);

    window_maximum(arr, size, k);

    return 0;
}
```

Test Cases

Test Case	Array	k	Output
1	1 3 -1 -3 5 3 6 7	3	3 3 5 5 6 7
2	1 2 3 4 5	2	2 3 4 5
3	1 2 3	4	-1
4	4 2 8 1 6	2	4 8 8 6

Sample Output 1



Working

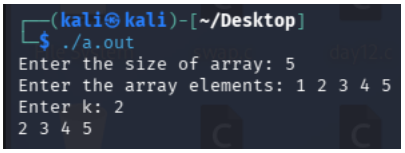
The sliding windows are:

```
1   3  -1        → Maximum = 3
3  -1  -3        → Maximum = 3
-1 -3   5        → Maximum = 5
-3  5   3        → Maximum = 5
5   3   6        → Maximum = 6
3   6   7        → Maximum = 7
```

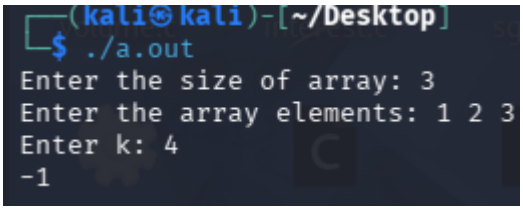
Therefore:

```
3 3 5 5 6 7
```

Sample Output 2



Sample Output 3 – Invalid k



Result

Thus, the C program to **find the maximum element in every sliding window of size `k` using pointers and a nested-loop approach** was implemented and executed successfully.